STUDENT MANAGEMENT SYSTEM

CSE 221: DATA STRUCTURE

Submitted by :

	Name	ID	Section	Intake
1	Abida Yeasmin Asifa	20255103182	5	55
2	Sanjidah Afrin	20255103185	5	55
3	Humayra Hossan Jeba	20255103166	5	55

Supervised by:
Talimul Bari Shreshtho
Lecturer,
Department of CSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
BANGLADESH UNIVERSITY OF BUSINESS AND TECHNOLOGY (BUBT)

29th, April, 2026

Abstract

This report presents the design and implementation of a console-based Student Management System developed in the C programming language. The system provides a complete suite of record management operations — adding, displaying, searching, updating, and deleting student entries — through a simple numbered menu interface that requires no graphical environment or external dependencies.

Data Structures

The project combines three fundamental data structures within a single program. A fixed-size array serves as the primary record store, offering O(1) positional access and straightforward sequential iteration. A stack, implemented as an array with a top pointer, captures deleted records and enables a one-step undo operation using Last-In First-Out (LIFO) semantics. A singly linked list with front-insertion maintains a chronological history of recently added students, ensuring the most recent entry is always displayed first.

Key Features

Beyond standard CRUD functionality, the system includes an undo-delete mechanism that restores the last removed record instantly, and a recent-students view that traverses the linked list to show addition history. All input is handled via scanf with careful format specifiers to correctly capture multi-word names alongside numeric fields.

Conclusion

The project successfully demonstrates how arrays, stacks, and linked lists each contribute distinct capabilities to a real application. It reinforces core C concepts including struct definition, pointer manipulation, dynamic memory allocation with malloc, and modular function-based design. The system is fully compilable with GCC using the provided Makefile and serves as a practical foundation for future enhancements such as file persistence and dynamic memory resizing.

Keywords: C Programming, Array, Stack, Linked List, CRUD, Data Structures

Table of Contents

Abstract —----------(i)

Chapter 1 | Introduction 1----------pages 1-2

1.1 Problem Statement

1.2 Objectives

1.3 Flow of the Project

1.4 Organization of the Report

Chapter 2 | Background 4—----------pages 2

2.1 Existing System Analysis

2.2 Supporting Literatures

Chapter 3 | System Analysis & Design 7 —----------pages 3-9

3.1 Technology & Tools

3.2 Model & Diagram

3.2.1 Development Model
3.2.2 Use Case Diagram
3.2.3 Context Level Diagram
3.2.4 Data Flow Diagram
3.2.5 Database Schema
3.2.6 Algorithms / Flowchart

3.3 Data Structure Justification

3.3.1 Array — Primary Record Store
3.3.2 Stack — Undo Delete
3.3.3 Singly Linked List — Recent Students History

Chapter 4 | Implementation 16 —---------- pages 10-11

4.1 Interface Design / Front-End

4.2 Back-End

4.3 Modules

Chapter 5 | User Manual 19 —----------pages 12-14

5.1 System Requirements

5.1.1 Hardware Requirements
5.1.2 Software Requirements

5.2 User Interfaces

5.2.1 Main Menu

5.2.2 Student Operations

5.2.3 Sample Output

Chapter 6 | Conclusion 25—----------pages 14-15

6.1 Conclusion

6.2 Limitation

6.3 Future Works

References 27—----------pages 16

Appendix A 28—----------pages 17-18

Chapter 1| Introduction

1.1 Problem Statement

Managing student records manually is time-consuming and error-prone. Educational institutions require a reliable system to store, retrieve, and maintain student data efficiently. This project addresses the need for a simple, lightweight record management solution that demonstrates core data structure concepts while solving a practical problem — storing and manipulating student records without relying on any external database or graphical interface.

1.2 Objectives

- Implement a fully functional CRUD (Create, Read, Update, Delete) student record system in C.
- Demonstrate the practical use of arrays for fixed-size record storage.
- Implement a stack to support an undo-delete feature.
- Use a singly linked list to maintain and display a history of recently added students.
- Provide a clean, menu-driven console interface for user interaction.

1.3 Flow of the Project

The program starts at **main()** and enters an infinite loop presenting an 8-option menu. The user selects an option and the corresponding function executes. The cycle repeats until the user chooses Exit. Data flows from user input through **scanf()** into the Student struct, stored in the array and simultaneously recorded in the linked list. Delete operations divert data to the stack for potential undo recovery.

1.4 Organization of the Report

This report is organized into six chapters. Chapter 1 introduces the problem and objectives. Chapter 2 reviews the background and existing systems. Chapter 3 covers system analysis and design including diagrams and data structures. Chapter 4 details the implementation. Chapter 5 provides the user manual. Chapter 6 concludes with limitations and future work.

Chapter 2 | Background

2.1 Existing System Analysis

Most student management systems today are either full database-backed web applications (such as those built with MySQL and PHP) or desktop applications with graphical interfaces. While powerful, these systems are heavy, require installation, and are not suitable for demonstrating core data structure concepts in an academic setting.

This project intentionally avoids external libraries and databases, relying purely on C's built-in capabilities. This makes it an ideal teaching tool — every operation is transparent and directly tied to a specific data structure concept.

2.2 Supporting Literatures

The design of this system is grounded in standard data structure theory:

- **Arrays** — provide O(1) random access and are the simplest form of contiguous memory storage, as described in Data Structures and Algorithm Analysis in C by Mark Allen Weiss.
- **Stacks** — follow Last-In First-Out (LIFO) semantics and are classically used for undo/redo mechanisms, backtracking, and function call management.
- **Linked Lists** — offer dynamic front-insertion in O(1) time and are well suited for maintaining ordered histories without shifting elements, as covered in The C Programming Language by Kernighan and Ritchie.

Chapter 3 | System Analysis & Design

3.1 Technology & Tools

Tool / Technology	Purpose
C Language (C99)	Core programming language
GCC Compiler	Compiling the source code
Makefile	Build automation (make / make run)
VS Code / Code::Blocks	Development environment
GitHub	Version control and project submission
Terminal / Command Prompt	Running the compiled program

3.2 Model & Diagram

3.2.1 Development Model

This project follows the **Waterfall Model** — a sequential development approach where each phase must be completed before the next begins. The phases were: Requirements → Design → Implementation → Testing → Deployment. This model was chosen because the project scope was clearly defined from the start with fixed requirements, making a linear approach appropriate.

3.2.2 Use Case Diagram

Actor: User (Student / Administrator)

Use Cases:

- Add Student Record
- Display All Students
- Search Student by ID
- Update Student Marks
- Delete Student Record
- Undo Last Delete
- View Recently Added Students
- Exit System

3.2.3 Context Level Diagram

The system has a single external actor — the User — who provides input (menu choice, student ID, name, marks) and receives output (records, confirmation messages, error notices) through the console. The Student Management System sits between input and output as a black box that processes all operations internally.

```
                    ┌──────────────────────────┐
                    │                          │
  User ────────────►│  Student Management System  │──────────► Console Output
  (Input: ID,       │                          │              (Records,
Messages)
   Name, Marks,     └──────────────────────────┘
   Menu Choice)
```

3.2.4 Data Flow Diagram

```
User Input
    |
    ▼
[Menu Selection]
    |
    ├──► Add ──────────► [Array Store] ──────────► [Linked List]
    |
    ├──► Display ──────► [Array Store] ──────────► Console Output
    |
    ├──► Search ───────► [Array Store] ──────────► Console Output
    |
    ├──► Update ───────► [Array Store] ──────────► Console Output
    |
    ├──► Delete ───────► [Stack] ◄──────────────── [Array Store]
    |
    ├──► Undo ─────────► [Stack] ──────────────► [Array Store]
    |
    └──► Recent ───────► [Linked List] ──────────► Console Output
```

User input enters through the menu selection. Based on choice, data flows to one of three internal stores:

Operation	Source	Destination
Add	User Input	Array + Linked List
Display	Array	Console Output
Search	Array	Console Output
Update	Array	Array (in-place)
Delete	Array	Stack (undo backup)
Undo	Stack	Array (restored)
Recent	Linked List	Console Output

3.2.5 Database Schema

Since this is an in-memory system, data is stored using C structures. The schema is:

Field	Data Type	Description
id	int	Unique student identifier
name	char[50]	Full name (up to 50 characters)
marks	float	Academic marks (e.g. 85.50)

Three storage containers are used: **students[100]** (primary array), **stackArr[100]** (undo stack), and **head** (linked list pointer).

3.2.6 Algorithms / Flowchart

Add Student Algorithm:

```
START

  IF count == MAX → print "Database Full", STOP

  INPUT id, name, marks

  students[count] = new record

  count++

  prepend to linked list

  print "Student Added Successfully"

END
```

Delete with Undo Algorithm:

```
START
  INPUT id to delete
  FOR i = 0 to count:
    IF students[i].id == id:
      push(students[i]) to stack
      SHIFT all elements left from i
      count--
      print "Deleted Successfully"
      STOP
  print "Student Not Found"
END
```

3.3 Data Structure Justification

This section addresses three questions for each data structure: **(1) Why it was chosen**, **(2) The rationale for that feature**, and **(3) Problems that would arise if a different structure had been used.**

3.3.1 Array — Primary Record Store

Why chosen: The array was chosen as the primary storage for student records because the maximum capacity (100) is known at compile time, and all CRUD operations need direct positional access. Arrays store elements contiguously in memory, giving O(1) access by index — ideal for iteration in displayStudents() and direct modification in updateStudent().

Rationale: Add, Display, Search, and Update all require scanning or directly addressing a specific slot. The counter variable **count** always points to the next free slot, so insertion is O(1). The deleteStudent() left-shift is O(n) but acceptable for a dataset capped at 100.

Problems if different structure used: A **Linked List** would require O(n) traversal to reach the i-th student — no random access. A **Stack** allows only top-element access, making search, update, and full display impossible without completely emptying and rebuilding the structure.

3.3.2 Stack — Undo Delete

Why chosen: The Stack was chosen because undo is inherently Last-In First-Out. When a user deletes a record it is pushed; when they press Undo the most recently deleted record is popped and restored. This maps perfectly to stack semantics with zero additional logic.

Rationale: Undo systems in virtually all software — text editors, spreadsheets, file managers — are implemented using stacks. The most recent action is always reversed first. The stack makes this a two-line implementation: push() on delete, pop() on undo.

Problems if different structure used: A **Queue** would restore the oldest deleted record first (FIFO) — incorrect undo behavior. An unordered **Linked List** would need extra logic to track which node was inserted last — complexity the stack handles automatically. Without a top pointer, a plain array backup would require scanning the whole area to find the last deletion.

3.3.3 Singly Linked List — Recent Students History

Why chosen: A singly linked list with front-insertion was chosen because the recent-students history must always show the newest entry first. Prepending a new node at the head takes O(1) regardless of how many students exist. The list also grows dynamically using malloc — no fixed cap on history depth.

Rationale: Every time addStudent() runs, the same record is simultaneously prepended to the linked list via addRecent(). Head-to-tail traversal then produces reverse-chronological order automatically — no sorting required.

Problems if different structure used: An **Array** inserting at index 0 would shift all existing elements right — O(n) per addition instead of O(1). A **Stack** cannot display all records without popping and destroying the history. A **Queue** would show oldest-first order, which contradicts the 'recently added' intent.

Chapter 4 | Implementation

4.1 Interface Design / Front-End

4.1 Interface Design / Front-End

The system uses a text-based console interface. The main menu is displayed inside a `while(1)` loop and re-renders after every operation. Visual separation is achieved using a `line()` function that prints a row of dashes.

```
---------------------------------------
 STUDENT MANAGEMENT SYSTEM
---------------------------------------
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Undo Delete
7. Recent Students
8. Exit
---------------------------------------
Enter Choice:
```

4.2 Back-End

The back-end consists entirely of C functions operating on three global data structures:

- **students[MAX]** — the primary array holding all active records
- **stackArr[MAX]** — the undo stack storing deleted records
- **head (Node*)** — the head pointer of the linked list for recent history

All data manipulation happens in memory. There is no file system or network interaction.

4.3 Modules

Function	Data Structure	Purpose
addStudent()	Array + Linked List	Add a new student record
displayStudents()	Array	Print all records
searchStudent()	Array	Find a record by ID
updateStudent()	Array	Modify marks in-place
deleteStudent()	Array + Stack	Remove and back up to stack
undoDelete()	Stack	Restore last deleted record
recentStudents()	Linked List	Show addition history
push() / pop()	Stack	Stack push and pop operations
addRecent()	Linked List	Prepend a node to the list
line()	—	Print UI separator line

Chapter 5 | User Manual

5.1 System Requirements

5.1.1 Hardware Requirements

Component	Minimum Specification
Processor	Any x86 / x64 CPU
RAM	256 MB
Storage	1 MB free disk space
Display	Any terminal or command prompt window

5.1.2 Software Requirements

Software	Details
Operating System	Windows 10+ / Linux / macOS
C Compiler	GCC (MinGW on Windows, built-in on Linux/macOS)
Build Tool	Make (optional, used with Makefile)
Terminal	CMD, PowerShell, Bash, or Zsh

5.2 User Interfaces

5.2.1 Main Menu

On launch, the user is presented with the numbered menu. Enter a number between 1–8 and press Enter to execute the corresponding operation.

5.2.2 Student Operations

Adding a student:

```
Enter ID: 101
Enter Name: Ahmed Rahman
Enter Marks: 85.5
Student Added Successfully!
```

Searching a student:

```
Enter ID to Search: 101
Student Found!
ID: 101
Name: Ahmed Rahman
Marks: 85.50
```

Deleting and undoing:

```
Enter ID to Delete: 101
Deleted Successfully!

[Select option 6]
Undo Successful!
```

5.2.3 Sample Output — Recent Students

```
--------------------------------------
Recently Added Students
--------------------------------------
ID: 103
Name: Fatima Hossain
Marks: 91.00
--------------------------------------
ID: 102
Name: Mehedi Islam
Marks: 78.50
--------------------------------------
ID: 101
Name: Ahmed Rahman
Marks: 85.50
--------------------------------------
```

Chapter 6 | Conclusion

6.1 Conclusion

The Student Management System successfully fulfils all its stated objectives. It demonstrates how arrays, stacks, and linked lists each contribute distinct capabilities within a single C program. The array provides fast indexed access for the main record store, the stack enables a user-friendly undo feature, and the linked list efficiently tracks insertion history without disturbing the primary data.

The system is fully compilable with GCC using the provided Makefile, satisfying the requirement that the project be runnable. All team members are listed as GitHub collaborators, fulfilling the submission requirements for the extra marks criteria.

6.2 Limitation

• Data is stored in memory only — all records are lost when the program exits (no file persistence). • The array has a fixed capacity of 100 student records.

• Only the marks field can be updated; name and ID cannot be changed after entry.

• There is no duplicate ID validation when adding a new student.

• The undo feature only reverses the most recent delete with no multi-step redo support. • The system is single-user and does not support concurrent access.

6.3 Future Works

• Add file I/O using fopen() / fwrite() to persist records across sessions.

• Replace the fixed array with a dynamically allocated structure for unlimited capacity. • Implement sorting by name, ID, or marks using qsort().

• Add input validation and duplicate ID checking.

• Extend undo/redo to a full history stack using a doubly linked list.

• Build a graphical interface using a library such as GTK or ncurses.

References

1. Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall. 2. Weiss, M. A. (2012). Data Structures and Algorithm Analysis in C (2nd ed.). Pearson.

3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

4. GNU Project. (2024). GCC, the GNU Compiler Collection. https://gcc.gnu.org

Appendix A — Source Code

Full source code of **student.c**:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 100

typedef struct Student {
    int id;
    char name[50];
    float marks;
} Student;

Student students[MAX];
int count = 0;

Student stackArr[MAX];
int top = -1;

typedef struct Node {
    Student data;
    struct Node *next;
} Node;

Node *head = NULL;
void line() {
printf("----------------------------------------\n"); }
```

```
void push(Student s) { if (top < MAX-1) stackArr[++top] = s; }
Student pop()        { return stackArr[top--]; }
void addRecent(Student s) {
    Node *n = (Node *)malloc(sizeof(Node));
    n->data = s; n->next = head; head = n;
}
/* ... full function implementations as per source file ... */
int main() {
    int choice;
    while (1) {
        line(); printf(" STUDENT MANAGEMENT SYSTEM\n"); line();
        printf("1. Add Student\n2. Display Students\n3. Search
Student\n");
        printf("4. Update Student\n5. Delete Student\n6. Undo
Delete\n");
        printf("7. Recent Students\n8. Exit\n");
        line(); printf("Enter Choice: "); scanf("%d", &choice);
        switch(choice) {
            case 1: addStudent();     break;
            case 2: displayStudents();break;
            case 3: searchStudent();  break;
            case 4: updateStudent();  break;
            case 5: deleteStudent();  break;
            case 6: undoDelete();     break;
            case 7: recentStudents(); break;
            case 8: printf("Program Ended.\n"); exit(0);
            default: printf("Invalid Choice!\n");
        }
    }
    return 0;
}
```

Full source code is available in the GitHub repository.